The Shift Cipher - Simplified Runnable Version

Sections 4–8 of the Chapter 1 tutorial (Listings 1-1 to 1-4, assembled) • 82 lines • one file • no imports

This is the same program as Sections 4 to 8 of the Chapter 1 tutorial, rewritten so that a third-year student can read every line without stopping. Save it as **`shift_cipher_simple.py`** and run it with **`python shift_cipher_simple.py`**. It needs nothing installed not even the `string` module.

What was simplified, and why

The book's version	This version	Why it is easier
import string + string.ascii_uppercase	ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"	One less import, and you can see the alphabet you are working with
Two functions, encode() and decode()	One function, convert()	They were always the same function; now that is obvious
message.upper() called at every menu branch	.upper() done once, inside convert()	Impossible to forget it and get an unchanged message back
sorted(subst.items()) with generator expressions, _ unpacking and " ".join()	A plain for loop building two strings	No generator expressions or underscore idiom needed
try / except ValueError / else with raise	answer.isdigit() and a range check	No exception handling required to understand the program
Returns a string, caller prints it	show_table() prints directly	One step instead of two

> **NOTHING WAS REMOVED**
>
> All five menu options still work, both tables are still built in one loop, the modulus still produces the wrap-around, and spaces and punctuation still pass through untouched. This is a simpler road to the same destination, not a shorter destination.

Part 1 - The alphabet (lines 1–5)

```
1  # shift_cipher_simple.py
2  # A simple shift (Caesar) cipher.
3  # Chapter 1 - Practical Cryptography in Python (Nielson & Monson)
4
5  ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

Line(s)	What that line actually does
1 – 3	Comments. Python ignores every line that begins with #. They are here so that anyone opening the file knows what it is.
4	A blank line. Python ignores it too; it is purely for readability.
5	The 26 capital letters, stored once in a variable we can reuse everywhere. The name is in CAPITALS because Python programmers write constants that way it is a signal to the reader that this value never changes.

Part 2 - Building the two tables (lines 8–16)

This replaces **Listing 1-1** (Section 4 of the tutorial).

```
8  def make_tables(shift):
9      encode_table = {}
10     decode_table = {}
11     for i in range(26):
12         plain = ALPHABET[i]
13         cipher = ALPHABET[(i + shift) % 26]
14         encode_table[plain] = cipher
15         decode_table[cipher] = plain
16     return encode_table, decode_table
```

Line(s)	What that line actually does
8	Defines the function. `shift` is how many places each letter moves it is the KEY. Because it is a parameter and not a fixed number, the user can change it later.
9	Creates an empty dictionary for **plain letter → cipher letter**. The `{}` means “empty dictionary”.
10	Creates a second empty dictionary for the opposite direction, **cipher letter → plain letter**.
11	Repeats the block below 26 times, with `i` taking the values 0, 1, 2 … 25. We count positions, not letters, because a shift is defined by position.
12	Picks the letter at position `i`. When `i` is 0 this is “A”; when it is 25 it is “Z”.
13	THE KEY LINE. `i + shift` moves along the alphabet, and `% 26` wraps it back to the start when it runs off the end. Without `% 26`, shifting “Z” (position 25) would ask for position 26, which does not exist, and Python would stop with an `IndexError`.
14	Stores the forward pair, for example `encode_table["A"] = "B"`.
15	Stores the same pair backwards, for example `decode_table["B"] = "A"`. Note the key and the value have simply swapped places that is all “decoding” means here.

16	Hands BOTH tables back at once, so the caller can write `a, b = make_tables(1)` in a single line.

TRACE LINE 13 BY HAND — it takes twenty seconds

With shift = 1: i = 0 → (0+1) % 26 = 1 → “B”. i = 24 → (24+1) % 26 = 25 → “Z”. i = 25 → (25+1) % 26 = 0 → “A”.

That last row is the wrap-around. Once you have seen it happen with real numbers, the modulus stops being mysterious.

Part 3 - Converting a message (lines 19–26)

This replaces **Listing 1-2** (Section 5). In the book this was two functions; here it is one, because encoding and decoding really are the same operation.

```
19  def convert(message, table):
20      result = ""
21      for letter in message.upper():
22          if letter in table:
23              result = result + table[letter]
24          else:
25              result = result + letter
26      return result
```

Line(s)	What that line actually does
19	Two inputs: the `message` to change, and the `table` to change it with. Passing the table in is the whole trick give it `encode_table` and it scrambles, give it `decode_table` and it unscrambles.
20	Starts with an empty string. We will glue the answer together one character at a time.
21	**`.upper()`** turns “hallo” into “HELLO” first, because our tables only contain capital letters. Looping over a string in Python gives you its characters one by one, so no counting or indexing is needed.
22	Asks: is this character one of the 26 letters in the table? For “H” yes; for a space, a comma or a digit, no.
23	If yes, glue on the replacement letter from the table.
24 – 25	If no, glue on the character exactly as it was. THIS is what keeps spaces and punctuation intact the behaviour you saw in the mapping tables.
26	Gives the finished string back to whoever called the function.

Part 4 - Printing a table (lines 29–36)

This replaces **Listing 1-3** (Section 6).

```
29  def show_table(table):
30      top = ""
31      bottom = ""
32      for letter in sorted(table):
33          top = top + letter + " "
34          bottom = bottom + table[letter] + " "
35      print(top)
36      print(bottom)
```

Line(s)	What that line actually does
29	Takes any table. Because it makes no assumption about direction, the same function prints the encoding table and the decoding table equally well.
30 – 31	Two empty strings: one for the row on top, one for the row underneath.

32	`sorted(table)` gives the table's keys in alphabetical order, A, B, C … Z. Without `sorted` the letters could come out in any order and the two rows would be unreadable.
33	Adds this letter and a space to the top row.
34	Adds the letter it maps to, plus a space, to the bottom row. The spaces are what make the two rows line up in columns on screen.
35 – 36	Prints the two rows, one above the other.

what lines 35–36 produce, for shift = 1

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
B C D E F G H I J K L M N O P Q R S T U V W X Y Z A
```

Part 5 - The menu program (lines 39–82)

This replaces **Listing 1-4** (Section 7). It is the part the user actually interacts with.

```
39 if __name__ == "__main__":
40     shift = 1
41     encode_table, decode_table = make_tables(shift)
42
43     while True:
44         print()
45         print("SHIFT CIPHER   (current shift =", shift, ")")
46         print("1. Show the tables")
47         print("2. Encode a message")
48         print("3. Decode a message")
49         print("4. Change the shift")
50         print("5. Quit")
51
52         choice = input(">> ")
```

Line(s)	What that line actually does
39	Runs the menu only when you execute this file directly. If another program *imports* the file to reuse `convert()`, the menu stays quiet. You will need exactly this when you write the brute-force attacker in Section 9.
40	The starting key. The user can change it later with option 4.
41	Builds both tables once, before the loop starts.
43	`while True` means “repeat forever”. The only way out is the `break` on line 79.
44	`print()` with nothing inside it just prints a blank line, so each menu is separated from the last.
45	Shows the key that is currently in force. Always displaying it prevents a great deal of confusion.
46 – 50	The five options.
52	`input()` prints `>>` and waits for the user to type. IMPORTANT: it always gives back **text**, never a number which is why every test below compares with `"1"` in quotes, not `1`.

```
54         if choice == "1":
55             print("Encoding table:")
56             show_table(encode_table)
57             print("Decoding table:")
58             show_table(decode_table)
59
60         elif choice == "2":
61             message = input("Message to encode: ")
62             print("Encoded:", convert(message, encode_table))
63
64         elif choice == "3":
65             message = input("Message to decode: ")
66             print("Decoded:", convert(message, decode_table))
```

Line(s)	What that line actually does

54 – 58	Option 1 prints both tables side by side, which is the quickest way to convince yourself that one really is the reverse of the other.
60 – 62	Option 2 asks for a message and prints the scrambled version. Note it passes **encode_table**.
64 – 66	Option 3 is the same three lines with one word changed: it passes **decode_table**. Compare these two blocks carefully that one word is the entire difference between encoding and decoding.

```
68          elif choice == "4":
69              answer = input("New shift (1 to 25): ")
70              if answer.isdigit() and 1 <= int(answer) <= 25:
71                  shift = int(answer)
72                  encode_table, decode_table = make_tables(shift)
73                  print("Shift is now", shift)
74              else:
75                  print("Please type a whole number from 1 to 25.")
76
77          elif choice == "5":
78              print("Bye!")
79              break
80
81          else:
82              print("Sorry, I do not know option", choice)
```

Line(s)	What that line actually does
68 – 69	Option 4 asks for a new key, reminding the user what the current one is.
70	`.isdigit()` checks that the user typed only digits, so `int()` on the next line cannot fail. The range check keeps the shift between 1 and 25, because 0 and 26 would leave the message unchanged.
71 – 72	Saves the new key AND rebuilds both tables. **Line 72 is the one students forget most often** change `shift` without rebuilding and the program silently keeps using the old key.
73	Confirms the change on screen.
74 – 75	If the input was not a sensible number, say so politely and change nothing. The program carries on with the key it already had.
77 – 79	Option 5. **`break`** is what finally escapes the `while True` loop and ends the program.
81 – 82	Anything that is not 1–5 lands here and gets a polite message instead of a crash. The loop then comes round again.

Running it - a real session

This is genuine output captured from the program, not an illustration. Type the same things and you must see the same results.

```
$ python shift_cipher_simple.py
SHIFT CIPHER   (current shift = 1 )
1. Show the tables
2. Encode a message
3. Decode a message
4. Change the shift
5. Quit
>> 2
Message to encode: hello world
Encoded: IFMMP XPSME

>> 4
New shift (1 to 25): 2
Shift is now 2

>> 2
Message to encode: hello world
Encoded: JGNNQ YQTNF

>> 3
Message to decode: JGNNQ YQTNF
Decoded: HELLO WORLD

>> 5
Bye!
```

LAB CHECKPOINT — your program is correct when all three of these hold

(a) With shift = 1, HELLO WORLD becomes IFMMP XPSME.

(b) With shift = 2, it becomes JGNNQ YQTNF.

(c) Encoding a message and then decoding the result with the SAME shift returns the original text exactly.

If (c) fails but (a) and (b) pass, you almost certainly left out line 72 the rebuild of the tables after changing the shift.

Three small experiments to try next

1. Delete % 26 from line 13, run the program and encode the letter Z. Read the error message carefully. Then put it back.
2. Delete .upper() from line 21 and encode “hallo” in lowercase. Explain in one sentence why nothing gets substituted.
3. Add lowercase letters to the tables so that “hallo” encodes to “ifmmp” instead of “IFMMP”. Hint: you only need to change make_tables().

Where this leads

Section 9 of the tutorial breaks this cipher. Because of line 39, you can reuse the code you just wrote without copying it put this in the same folder and run `python brute_force.py`:

brute_force.py

```
1  from shift_cipher_simple import make_tables, convert
2
3  ciphertext = "FANQADZAFFANQFTMFUEFTQCGQEFUAZ"
4
5  for shift in range(1, 26):
6      encode_table, decode_table = make_tables(shift)
7      print(shift, convert(ciphertext, decode_table))
```

Seven lines, and the entire key space of the cipher is gone. Look for the one line of output that reads as English.